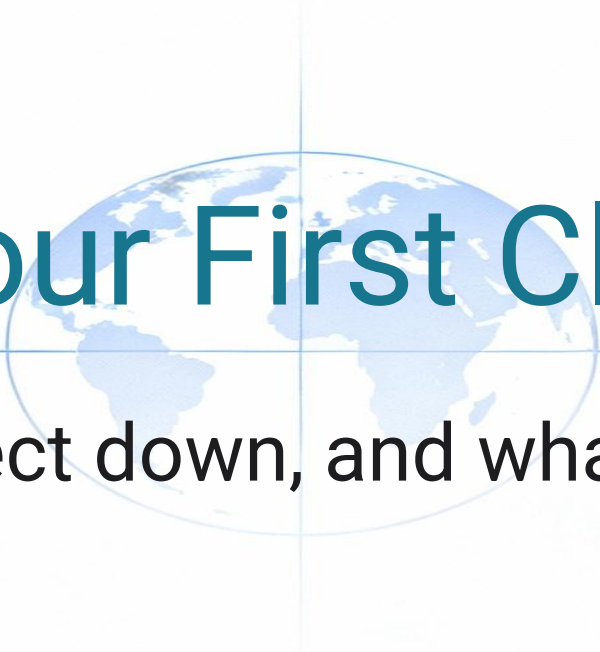




Your First Class

Writing an object down, and what a variable holds

Evis Plaku



A thing **knows** something and **does** something

- What it knows becomes fields. What it does becomes methods
- The moment it comes into existence is the constructor
- Today you write one down. The vocabulary is that small

State



Behavior



Request → Response

One description, many things

The class, one description

Account

accountNumber

ownerName

balance

holds no money

The objects, each with its own state

AC-1001	Ada Lovelace	500.00
AC-1002	Alan Turing	2500.00
AC-1003	Grace Hopper	1500.00



Fields, and every one of them private

- private is the wall from last week, now enforced by the compiler
- Outside this file, `account.balance -= 5000` does not compile
- double for money is fine here and wrong for a real bank

```
public class Account {  
  
    private String accountNumber;  
    private String ownerName;  
    private double balance;  
  
}
```



The constructor is **unlike any method** you have written

- Same name as the class, and no return type at all
- It runs once, when the object is created, and never again
- Now an Account can exist with something in it

```
public Account(String accountNumber,  
                String ownerName,  
                double openingBalance) {  
    this.accountNumber = accountNumber;  
    this.ownerName      = ownerName;  
    this.balance        = openingBalance;  
}  
  
Account ada = new Account(  
    "AC-1001", "Ada Lovelace", 500.00);
```



Leave out `this.` and nothing warns you

- `this.ownerName` is the field. `ownerName` alone is the parameter
- Without it the line assigns the parameter to itself
- You find it later, when the account prints a blank owner

```
// what you mean  
this.ownerName = ownerName;
```

```
// what you wrote  
ownerName = ownerName;
```

```
// the parameter is nearer, so both  
// names mean the parameter  
// the field stays empty  
// the compiler says nothing
```



One constructor, **not two**

- Most tutorials show a full constructor and an empty one
- The empty one makes an account with no number, no owner, no money
- A constructor exists to guarantee the object is usable from the start

```
// do not add this
public Account() {
}

// it produces a half-built thing
// that some later line is supposed
// to remember to finish

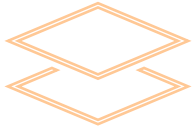
// sooner or later a line forgets
```



Methods that can say no

- The account holds the balance, so the account decides
- deposit has the same shape and the same power to refuse
- Both report what happened. Neither of them prints

```
public boolean withdraw(double amount) {  
    if (amount <= 0 || amount > balance)  
    {  
        return false;  
    }  
    balance -= amount;  
    return true;  
}
```



A domain class **does not talk to the user**

- Account does not know if it is used by a console, a page or a test
- A method that prints cannot be tested. That is week 10
- A method that prints cannot drive a window. That is week 13

```
// tempting, and wrong
public boolean withdraw(double a) {
    if (a > balance) {
        System.out.println("No funds");
        return false;
    }
    ...
}

// Account reports to the caller.
// The caller reports to the user.
```



Three getters, and no setBalance

- There is no setBalance, and there never will be
- It would hand back exactly the power that private took away
- The balance moves through deposit and withdraw, or it does not move

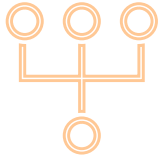
```
public double getBalance() {  
    return balance;  
}  
  
// setBalance(double b) {  
//     balance = b;  
// }  
  
// every setter you add is a rule  
// you decided not to enforce
```



toString is **four lines and a debugger**

- Without it, printing an object gives you `Account@6d06d69c` (or similar)
- `@Override` makes the compiler catch a misspelt `toString`
- Write EUR, not the symbol. The Windows console mangles it

```
@Override
public String toString() {
    return String.format(
        "Account %s (%s): %.2f EUR",
        accountNumber, ownerName,
        balance);
}
```



Every word the user reads is written in main

- Account never says anything. It answers, and main does the talking
- Three accounts, three separate balances, one description
- Nothing validates yet. Week 3 closes that on purpose

```
Account ada = new Account(
    "AC-1001", "Ada Lovelace", 500.00);

System.out.println(ada);

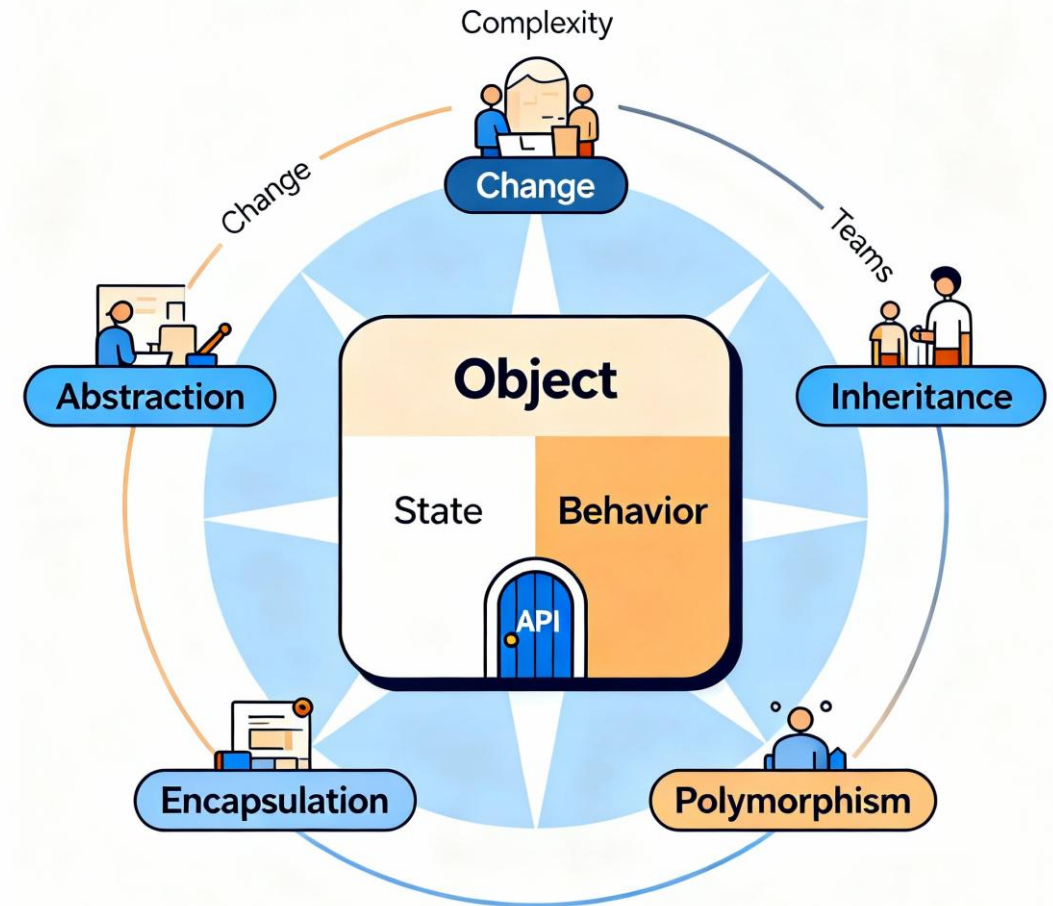
if (ada.withdraw(200.00)) {
    System.out.println("Accepted.");
} else {
    System.out.println("Not enough.");
}

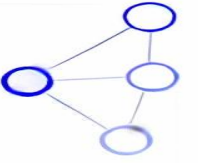
// Account AC-1001 (Ada Lovelace):
//   500.00 EUR
// Accepted.
```



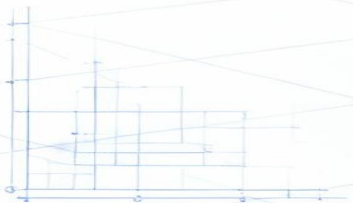
A class describes. An object exists

- The constructor is the moment one turns into the other
- Nothing should exist in a state you would not want to use
- Next: what a variable actually holds, and why it matters





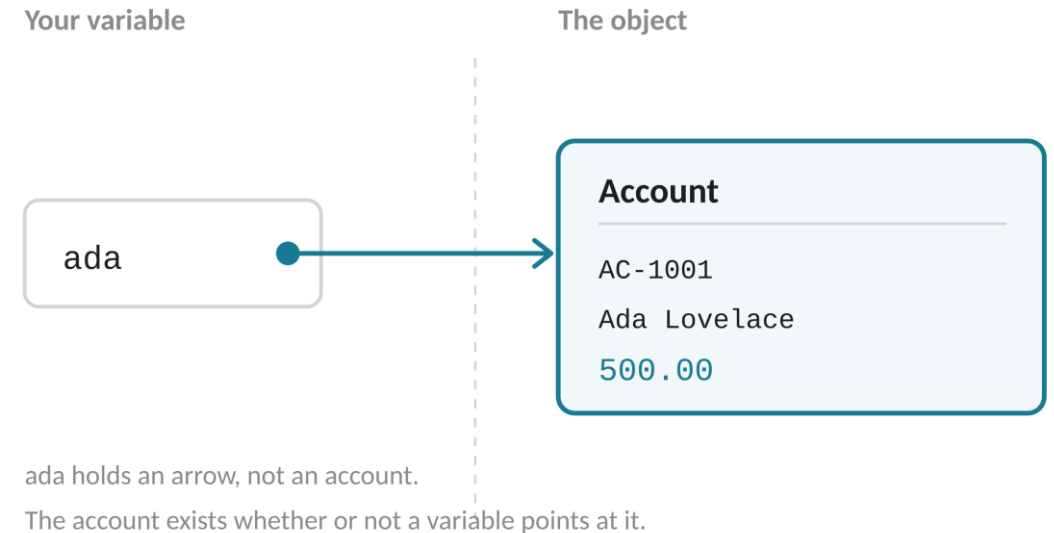
What a variable really holds





What **new** actually did

- It set aside space for an account somewhere and filled it in
- Then it produced a reference: the way to reach that space
- The account and the variable are not in the same place





Primitives are different

- int, double, boolean and char hold their value directly
- Everything else in Java holds a way to reach the value

A primitive holds the value

```
double balance  
500.00
```

the number is right there

Everything else holds a way to reach it

```
Account ada
```



```
Account AC-1001 500.00
```

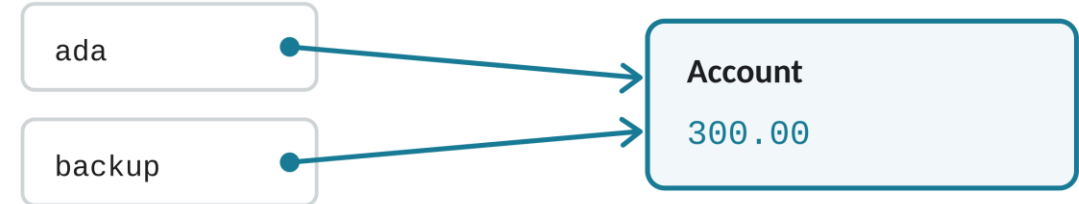
int, double, boolean, char hold values. Every object is reached by an arrow.



Two names, one account

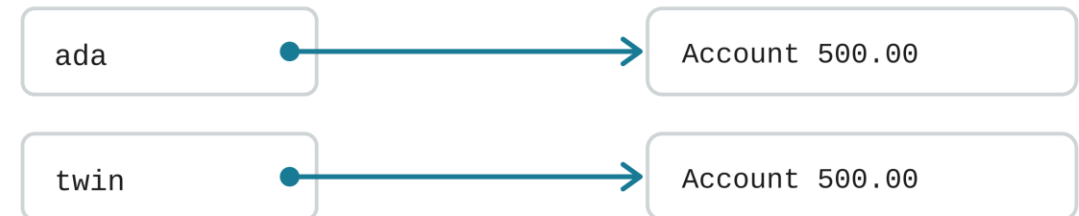
- backup = ada copies the *pointer*:
two pointers, one account
- Withdraw through either name and
the other one sees it
- Two new Account lines make two
accounts, however alike

Two names, one object



Withdraw through either name. There is only one balance to change.

Two objects that look alike



Same values, different things.
ada == twin is false.



== compares **arrows**, not contents

- It never looks at the values inside, however many fields match
- It answers a real question: are these the same object?
- For "the same account" you need equals, and that is week 5

```
Account ada    = new Account(...);
Account backup = ada;
Account twin    = new Account(...);

ada == backup;    // true
ada == twin;      // false

// same object?  not: same values?
```



This applies to **String** too

- String is an object, so `==` on strings compares arrows
- Java reuses identical literals, so `==` often appears to work
- It stops working the moment the text comes from a file

```
String a = new String("AC-1001");  
String b = new String("AC-1001");
```

```
a == b;           // false  
a.equals(b);      // true
```

```
String c = "AC-1001";  
String d = "AC-1001";  
c == d;           // often true
```

```
// use .equals for strings. Always.
```



null is not an empty account. It is the absence of one

- Asking it for a balance means following an arrow that does not exist
- The message names the variable and the method. Read all of it
- It comes from a field nobody set, or a search that found nothing

```
Account closed = null;  
closed.getBalance();
```

```
Exception in thread "main"  
java.lang.NullPointerException:  
Cannot invoke  
    "Account.getBalance()"   
    because "closed" is null
```



Java copied **the arrow**, not the account

- The method gets its own variable pointing at the same object
- So the fee lands on the real thing, and the change survives
- This surprises people who were told Java always copies



The arrow was copied. The account was not.



You can change the object. You cannot move the caller

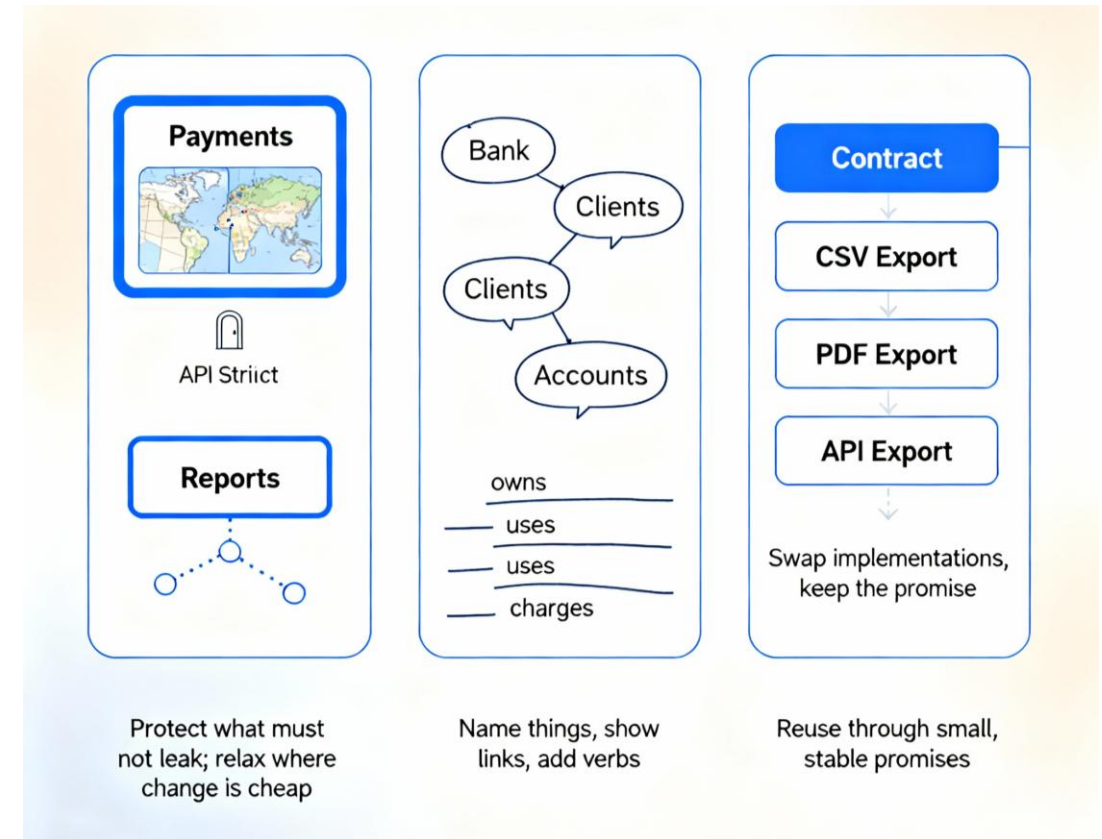
- applyFee changes the account it was handed, and that survives
- replace points its own arrow elsewhere, and the caller never moves
- Same rule as backup = ada, one slide earlier

```
static void applyFee(Account account) {  
    account.withdraw(5.00);  
}  
// ada is 5.00 lighter  
  
static void replace(Account account) {  
    account = new Account(...);  
}  
// ada is unchanged
```



Questions with no answer to look up

- backup = ada copies an arrow. How would you get a separate account?
- Two accounts with one number are not equal. Right default?
- Nobody points at the object made inside replace. What happens?



The recipe isn't the meal
One tells you how. The other you can eat

